



TaskFlow API

Ejemplo de documentación de una aplicación



AUTOR

Equipo de Documentación
docs@example.com

FECHA

15/01/2026

VERSIÓN

2.3

ORGANIZACIÓN

Acme Corp

Índice

Introducción.....	3
Instalación	3
Configuración	3
Uso de la API	4
Con curl	4
Con Python	5
Con TypeScript	6
Modelo de datos.....	6
Despliegue con Docker	7
Códigos de respuesta	8

Introducción

TaskFlow es una aplicación ficticia de gestión de tareas con una API REST. Este documento muestra cómo se ve la **documentación de una aplicación** con muchos bloques de código: cada bloque indica su lenguaje y se resalta con los colores de la sintaxis.

📘 Sobre este ejemplo

El estilo del resaltado se elige con `highlight_style` en la configuración (`pygments`, `tango`, `kate`, `monochrome`...). Los nombres de lenguaje habituales en Obsidian, como `shell` o `jsonc`, se reconocen automáticamente.

Para la arquitectura del servicio de notificaciones que usa esta API, consulta [la documentación técnica](#).

Instalación

Necesitas Python 3.11 o superior y Docker. Clona el repositorio y crea un entorno virtual:

```
git clone https://git.example.com/acme/taskflow.git
cd taskflow
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Arranca la base de datos y aplica las migraciones:

```
docker compose up -d db
alembic upgrade head
uvicorn taskflow.main:app --reload --port 8000
```

Configuración

La aplicación se configura con un fichero YAML. Todas las claves tienen un valor por defecto razonable.

```
# config/taskflow.yml
server:
  host: 0.0.0.0
  port: 8000
  workers: 4

database:
  url: postgresql://taskflow:secret@localhost:5432/taskflow
  pool_size: 10

auth:
  token_ttl_minutes: 60
  allowed_origins:
    - https://app.example.com
    - https://admin.example.com

features:
  notifications: true
  audit_log: false
```

Las respuestas de la API siempre son JSON. Este es el aspecto de una tarea:

```
{
  "id": "6f1c2a1e-9d5b-4c0e-8a55-2b1f0c9d7e34",
  "title": "Preparar la demo del jueves",
  "status": "in_progress",
  "priority": 2,
  "tags": ["demo", "cliente"],
  "assignee": { "id": 42, "name": "Ana Ejemplo" },
  "due_date": "2026-02-01",
  "done": false
}
```

Uso de la API

Con curl

Todas las peticiones llevan el token en la cabecera `Authorization`:

```
export TOKEN="tu-token-de-ejemplo"

# Crear una tarea
curl -X POST https://api.example.com/v1/tasks \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"title": "Revisar el informe", "priority": 1}'

# Listar las tareas pendientes
curl -s "https://api.example.com/v1/tasks?status=todo&limit=20" \
  -H "Authorization: Bearer $TOKEN" | jq '.items[] | .title'
```

Con Python

```
import os
import requests

API = "https://api.example.com/v1"
HEADERS = {"Authorization": f"Bearer {os.environ['TOKEN']}"}

def crear_tarea(titulo: str, prioridad: int = 3) -> dict:
    """Crea una tarea y devuelve el JSON de la respuesta."""
    resp = requests.post(
        f"{API}/tasks",
        json={"title": titulo, "priority": prioridad},
        headers=HEADERS,
        timeout=10,
    )
    resp.raise_for_status()
    return resp.json()

if __name__ == "__main__":
    tarea = crear_tarea("Escribir la documentación", prioridad=1)
    print(f"Creada {tarea['id']}: {tarea['title']}")
```

Con TypeScript

```
interface Task {
  id: string;
  title: string;
  status: "todo" | "in_progress" | "done";
  priority: number;
}

async function listTasks(status: Task["status"]): Promise<Task[]> {
  const res = await fetch(`https://api.example.com/v1/tasks?status=${status}`, {
    headers: { Authorization: `Bearer ${process.env.TOKEN}` },
  });
  if (!res.ok) {
    throw new Error(`Error ${res.status}: ${await res.text()}`);
  }
  const data = (await res.json()) as { items: Task[] };
  return data.items;
}

listTasks("todo").then((tasks) => console.log(`${tasks.length} pendientes`));
```

Modelo de datos

El esquema relacional es sencillo: usuarios, tareas y etiquetas.

```
CREATE TABLE users (
  id          SERIAL PRIMARY KEY,
  name        TEXT      NOT NULL,
  email       TEXT      UNIQUE NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE tasks (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title       TEXT      NOT NULL,
  status      TEXT      NOT NULL DEFAULT 'todo',
  priority    SMALLINT  NOT NULL DEFAULT 3,
  assignee    INTEGER REFERENCES users (id),
```

```
    due_date    DATE
);

-- Tareas vencidas por persona
SELECT u.name, COUNT(*) AS vencidas
FROM tasks t
JOIN users u ON u.id = t.assignee
WHERE t.status <> 'done' AND t.due_date < CURRENT_DATE
GROUP BY u.name
ORDER BY vencidas DESC;
```

Despliegue con Docker

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY taskflow/ taskflow/
COPY config/ config/

ENV TASKFLOW_CONFIG=/app/config/taskflow.yml
EXPOSE 8000
CMD ["uvicorn", "taskflow.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Si quieres automatizar el despliegue con integración continua, mira la [guía de Jenkins en Docker](#).

Códigos de respuesta

Código	Significado	Cuándo ocurre
200	OK	La petición se ha procesado
201	Creado	Se ha creado una tarea
401	No autorizado	Falta el token o ha caducado
404	No encontrado	La tarea no existe
422	Datos no válidos	Falta un campo obligatorio

Una línea muy larga se ajusta al ancho de la página en lugar de salirse del margen:

```
curl -X POST https://api.example.com/v1/tasks -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -H "X-Request-Id: 9f8b7c6d-5e4f-3a2b-1c0d-ef1234567890" -d '{"title": "Una tarea con un título bastante largo para comprobar el ajuste de línea", "priority": 1, "tags": ["ejemplo", "largo"]}'
```